

Функции

Функция — код (множество операторов), выполнение которого, возможно, потребуется повторять в различных местах страницы.

То есть, **описав** (объявив) функцию один раз, мы можем её **вызывать** из различных мест кода, и каждый раз будут выполняться операторы, составляющие **тело** функции.

После того как вызванная функция выполнилась (отработала), выполнение возвращается в точку, откуда она была вызвана, и далее выполняются операторы, следующие за вызовом функции.

Функция может получать **аргументы** — переменные, которые видны только в теле функции и которые получают конкретные значения при **вызове** функции. Функция может и не получать аргументов.

Функция может **возвращать результат** — значение любого типа, представляющее результат её работы. В качестве результата функция может возвращать только одно значение, либо не возвращать никакого значения.

Как назвать функцию, должна ли она получать аргументы, какие и в каком порядке, будет ли она возвращать результат и какой — решает разработчик.

Функции, во-первых, важны для **повторного использования кода**, то есть, единожды написав и отладив некий осмысленный (полезный) код, мы (или другой разработчик) можем этим кодом в дальнейшем пользоваться многократно, в этом или в других проектах. Во-вторых, функции позволяют делать код компактнее и яснее.

Синтаксис описания (объявления) функции:

```
function имяфункции(аргумент1, аргумент2, ...) {  
    операторы (тело функции)  
}
```

В теле функции можно использовать оператор возврата результата:

```
return возвращаемоеЗначение;
```

При такой записи оператор `return` прекращает выполнение функции и **возвращает результат**. Если выполнение функции требуется прекратить, а результат возвращать не требуется:

```
return;
```

Если выполнение кода в теле функции дошло до самого конца, а оператор `return` так и не встретился, выполнение функции также прекращается, без возврата значения.

По какой бы причине не прекратилось выполнение функции — далее выполняются операторы, которые следуют после вызова функции.

Синтаксис вызова функции:

```
имяФункции(значение1, значение2, ...)
```

При вызове функции, **переданные** функции значения присваиваются переменным-аргументам в функции, т.е. когда тело функции выполняется, переменные-аргументы имеют те значения, которые **переданы** функции при её вызове.

Если при описании функции аргументы не были указаны, т.е. функции аргументы не требуются, то и при вызове функции их указывать не нужно, однако, круглые скобки обязательны:

```
имяФункции()
```

Пример:

```
// описываем функцию получения количества дней в году
function getYearDays(year) {
    if ( year%4==0 )
        return 366;
    else
        return 365;
}
```

```
// при вызове getYearDays переменная-аргумент year получит значение 2013
console.log( 'В 2013 году дней: ' + getYearDays(2013) );
```

В 2013 году дней: 365

```
// при вызове getYearDays переменная-аргумент year получит значение 2012
console.log( 'В 2012 году дней: ' + getYearDays(2012) );
```

В 2012 году дней: 366

Область видимости переменных

Переменные-аргументы функции, а также любые переменные, описанные в теле функции обычным образом (например, `var имяПеременной=значение`), видны (доступны для обращения) только в теле этой функции, обращаться к ним из основной части кода или из тела других функций не получится (кроме особого случая — вложенные описания функции). Т.е. у них **локальная область видимости**.

Переменные, описанные обычным образом (например, `var имяПеременной=значение`) в основной части кода (т.е. в теге `script`, не в теле функции), доступны откуда угодно, т.е. у них **глобальная область видимости**.

У переменных, описанных без ключевого слова `var` (например, `имяПеременной=значение;`) всегда **глобальная область видимости**, где бы они ни были описаны.

Пример:

```
var a=3;
var g=100;

function f(m) {
    var a=5;
    var b=10;

    console.log("f видит m="+m);
    console.log("f видит a="+a);
    console.log("f видит b="+b);
    console.log("f видит g="+g);
}

f(51);

console.log("основной код видит a="+a);
console.log("основной код видит g="+g);
```

```
f видит m=51
f видит a=5
f видит b=10
f видит g=100
основной код видит a=3
основной код видит g=100
```